

Unit 2. Linux Commands and Utilities

2.1 Locating Commands, Internal & External Commands, Arguments, Options & Filenames, Online help

Internal & External Commands

The Linux system is command-based i.e things happen because of the commands that you key in. All UNIX commands are seldom more than four characters long.

They are grouped into two categories:

Internal Commands : Commands which are built into the shell. For all the shell built-in commands, execution of the same is fast in the sense that the shell doesn't have to search the given path for them in the PATH variable, and also no process needs to be spawned for executing it.

Examples: source, cd, fg, etc. time, kill, type, exit

How to get the list of Internal commands?

You can get only if you are in bash shell. Bash shell has a command called "help" which will list out all the built-in shell commands.

```
$ help
```

```
alias [-p] [name[=value] ... ]    bg [job_spec ...]
```

```
bind [-lpvsPVS] [-m keymap] [-f fi break [n]
```

```
builtin [shell-builtin [arg ...]] caller [EXPR]
```

```
case WORD in [PATTERN [| PATTERN]. cd [-L|-P] [dir]
```

```
command [-pVv] command [arg ...]  compgen [-abcdefgjkuv] [-o option
```

```
.....
```

```
.....
```

External Commands : **External commands** are the **commands** that are executed by the **kernel**. Commands which aren't built into the shell. When an external command has to be executed, the shell looks for its path given in the PATH variable, and also a new process has to be spawned and the command gets executed. External Command existed in any **PATH** directory like */bin, /usr/bin* etc. like *ls, sed, mv*. These *commands* having independent existence in */bin, /usr/bin*, or any other directory. These commands will have a

process id running for it. For example, when you execute the “cat” command, which usually is at /usr/bin, the executable /usr/bin/cat gets executed.

Examples: ls, cat etc.

How to find out whether a command is internal or external?

In addition to this you can also find out about a particular command *i.e* whether it is internal or external with the help of **type** command :

\$type cat

cat is /bin/cat

//specifying that cat is external type//

\$type cd

cd is a shell builtin

//specifying that cd is internal type//

What is the difference between internal and external commands?

Internal commands are commands that are already loaded in the system. They can be executed any time and are independent. On the other hand, external commands are loaded when the user requests for them. Internal commands don't require a separate process to execute them. External commands will have an individual process. Internal commands are a part of the shell while external commands require a Path. If the files for the command are not present in the path, the external command won't execute.

What is the difference between internal and external commands?

- The commands that are directly executed by the shell are known as internal commands. No separate process is there to run these commands.
- The commands that are executed by the kernel are known as external commands. Each command has its unique process id.

Refer following link for more information

http://linuxcommand.org/lc3_lts0060.php

Syntax of UNIX/ Linux Command Lines

UNIX/ Linux command lines can be simple, one-word entries like the **date** command. They can also be more complex: you may need to type more than the command name.

A UNIX command may or may not have *arguments*. An argument can be an option or a filename. The general format for UNIX commands is:

command *option(s)* *filename(s)*

There isn't a single set of rules for writing UNIX commands and arguments, but you can use these general rules in most cases:

- Enter commands in lowercase.
- *Options* modify the way in which a command works. Options are often single letters prefixed with a dash (-) and set off by any number of spaces or tabs. Multiple options in one command line can be set off individually (like **-a -b**), or, in some cases, you can combine them after a single dash (like **-ab**).

Some commands, including those on Linux systems, also have options made from complete words or phrases, like **--delete** or **--confirm-delete**. When you enter a command, you can use this option style, the single-letter options (which all start with a single dash), or both.

- The argument *filename* is the name of a file that you want to use. If you don't enter a filename correctly, you may get the response "*filename*: no such file or directory" or "*filename*: cannot open."

Some commands, like **telnet** and **who** (shown earlier in this chapter), have arguments that aren't filenames.

- You must type spaces between commands, options, and filenames.
- Options come before filenames.
- Two or more commands can be written on the same command line, each separated by a semicolon (;). Commands entered this way are executed one after another by the shell.

UNIX has a lot of commands! Don't try to memorize all of them. In fact, you'll probably need to know just a few commands and their options. As time goes on, you'll learn these commands and the best way to use them for your job. We cover some useful UNIX commands in later chapters.

Let's look at a sample UNIX command. The **ls** command displays a list of files. It can be used with or without options and arguments. If you enter:

% ls

a list of filenames will be displayed on the screen. But if you enter:

% ls -l

there will be an entire line of information for each file. The **-l** option (a dash and a lowercase letter “l”) modifies the normal output of the **ls** command and lists files in the long format. You can also get information about a particular file by using its name as the second argument. For example, to find out about a file called *chap1*, enter:

% ls -l chap1

Many UNIX commands have more than one option. For instance, **ls** has the **-a** (*all*) option for listing hidden files. You can use multiple options in either of these ways:

% ls -a -l

% ls -al

You must type one space between the command name and the dash that introduces the options. If you enter **ls-al**, the shell will say “ls-al: command not found.”

help Command

help command displays information about shell built-in commands. Here's the syntax for it:

// syntax for help command

\$help [-dms] [pattern ...]

The pattern specified in the syntax above refers to the command about which you would like to know and if it is matched with any shell built-in command then **help** give details about it and if it is not matched then **help** prints the list of help topics for your convenience. And the d, m and s here are options that you can use with the **help** command.

// using help

\$help help

help: help [-dms] [pattern...]

Display information about builtin commands.

Displays brief summaries of builtin commands. If PATTERN IS specified, gives detailed help on all commands matching PATTERN, otherwise the list of help topics is printed.

Options:

- d output short description for each topic
- m display usage in pseudo-manpage format
- s output only a short usage synopsis for each topic matching
PATTERN

Arguments:

PATTERN Pattern specifying a help topic

Exit Status:

Returns success unless PATTERN is not found or an invalid option is given.

/* so that's what help command

does telling everything

about the command and

helping you out */

Options for help command

- **-d option** : It is used when you just want to get an overview about any shell built-in command *i.e* it only gives short description.
- **-m option** : It displays usage in pseudo-manpage format.
- **-s option** : It just displays only a short usage synopsis for each topic matching.

Using help with options

- **Using -d** : This option just lets you know about what a command does without giving you details about its options and other stuff.
- // using help with -d
-
- **\$help -d help**
- help - Display information about builtin commands.
- **Using -s** : This option is when you just want to know about the syntax of a command.
- // using help with -s
-
- **\$help -s help**

help: help [-dms] [pattern ...]

....